pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

Feature Engineering on Financial Ratios

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

A feature is something you know, written as a column

A shape

The generator caps maturity at 10 years

An identity

Shared denominators give new ratios

A gap

A missing value carries information

The rule for the whole day

Anything a feature learns from data, it learns from the training rows of each fold, inside a pipeline.

Then two changes that add no column: the quantile transform and clipping.

Course 4's test rows stay closed on both files: every number today is cross-validated inside its training rows.

How a capped feature works

1. Spot the bend

the spread rises to 10 years, then stops

2. Write it down

min(years, 10), from each bond's own maturity

3. Keep in pipeline

no statistic is learned, so nothing can leak

4. Compare

cross-validated error with and without

You control

The cap, and where the function lives: inside the pipeline.

What comes out

One more column, and the error with and without it.

A feature that writes down a shape

```
oas = BASE_OAS[score] * (0.70 + 0.30 * min(years_to_maturity, 10) / 10) * ...
```

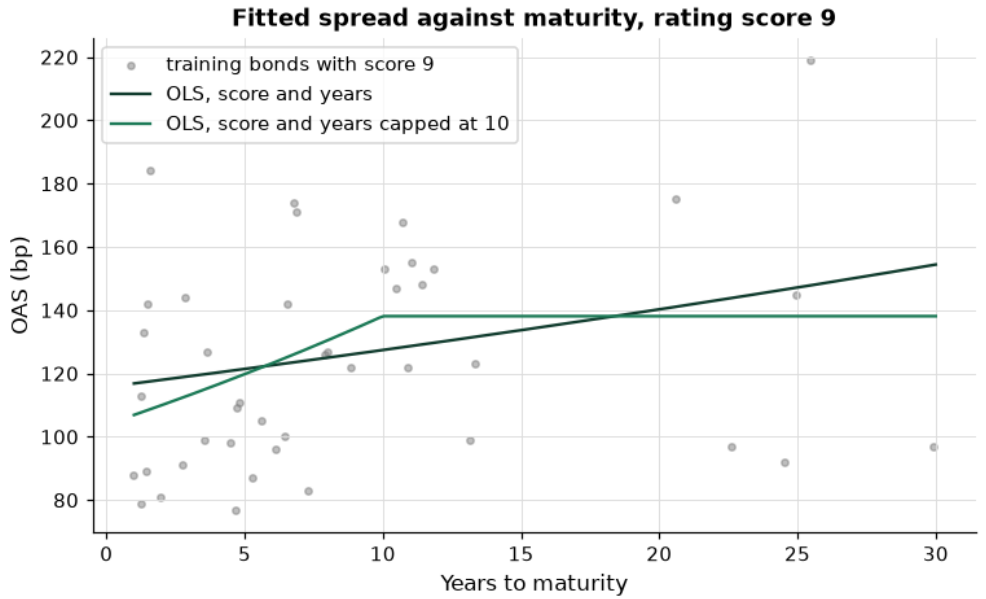
42.3 bp

OLS, score and years

40.9 bp

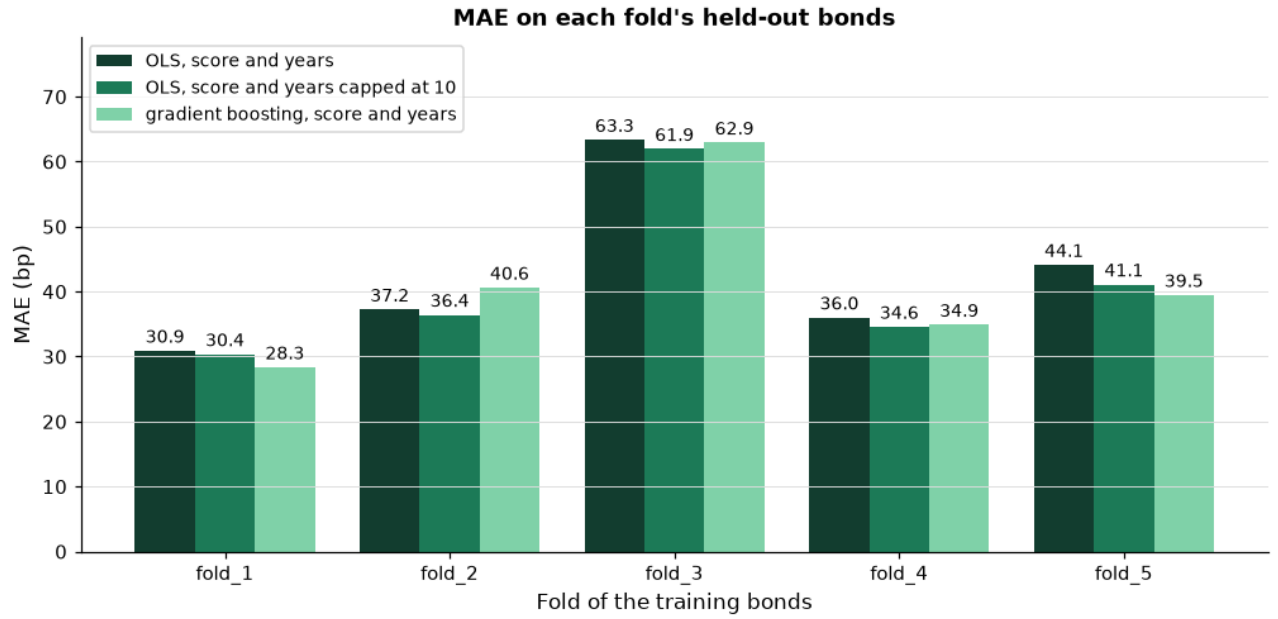
OLS, score and capped years

181 of the 615 training bonds have more than 10 years left. Cross-validated MAE, five folds of the training bonds.



Same model, same rows, one column written differently. The output is a fitted spread.

With the cap, OLS matches boosting



Mean of five folds

40.9 bp

OLS, capped years

41.2 bp

gradient boosting, raw years

42.3 bp

OLS, raw years

Within a fold the bars are close; across folds they move together. With the cap, two coefficients do what a hundred trees do.

How ratios from ratios work

1. Shared divisor

two ratios over total assets

2. Divide them

the shared part cancels

3. Guard division

zero or empty gives a missing value

4. Add at the front

of the pipeline, row by row

You control

Which pairs to divide, and what a zero denominator returns.

What comes out

New columns; the cross-validated score says whether they earn a place.

Ratios from ratios

New feature	Built as	Reads as
debt_to_equity	X2 / X10	total liabilities / equity
ebit_to_liabilities	X7 / X2	EBIT / total liabilities
short_term_share	X51 / X2	short-term share of liabilities

```
ensemblekit.safe_ratio(X_train["Attr7"], X_train["Attr2"])
```

X2 = 0 on 12 training rows

Plain division: 9 infinities, 5 missing. safe_ratio: 0 infinities, 14 missing.

A missing value, a model knows how to handle. An infinity, it does not.

4,680 training rows, 326 bankrupt (6.97 percent). Course 4 reads Attr k as the page's X k.

Ratios as features, 1968

Edward I. Altman, September 1968

"Financial Ratios, Discriminant Analysis and the Prediction of Corporate Bankruptcy" (not read for this course)

The Journal of Finance, volume 23, issue 4, pages 589 to 609

Its title puts financial ratios at the start of a model of company bankruptcy, where today puts them.

Source, read 5 October 2026: the Crossref record of the paper (DOI 10.1111/j.1540-6261.1968.tb00843.x). Bibliographic facts only; the paper itself was not read, and no claim is made about which ratios it used.

How missing values are handled

1. LightGBM: gaps

it learns which side they go

2. An imputer fills

the column's median, from its fitted rows

3. A missing count

gaps in the row, from the row alone

4. Score each

by the same five folds

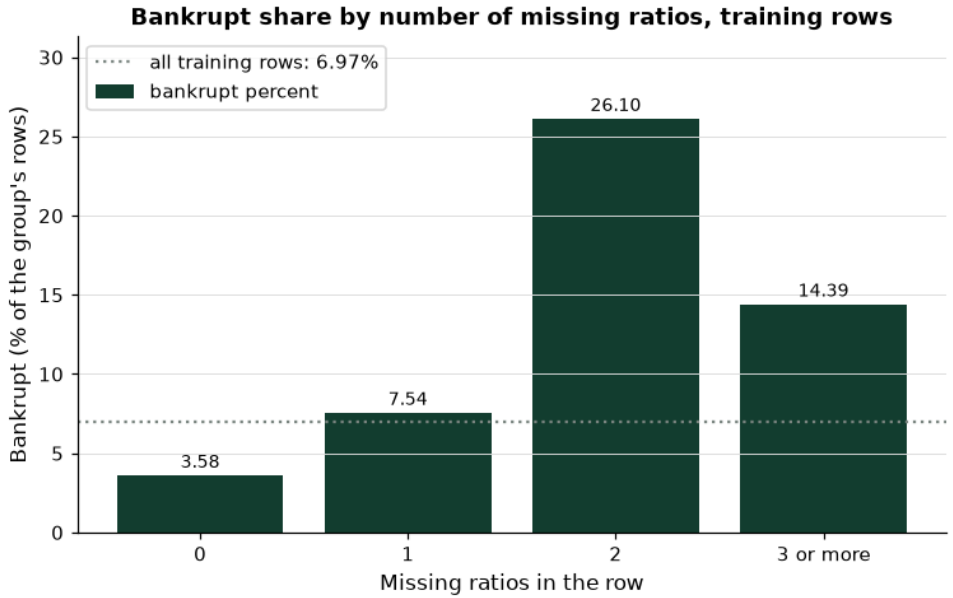
You control

The imputer's strategy, indicator columns, and the count.

What comes out

A table with no gaps or a model that handles them, and a score for each choice.

Missingness as a feature



0.7848

LightGBM, raw columns

0.7571

after a median imputer

0.7798

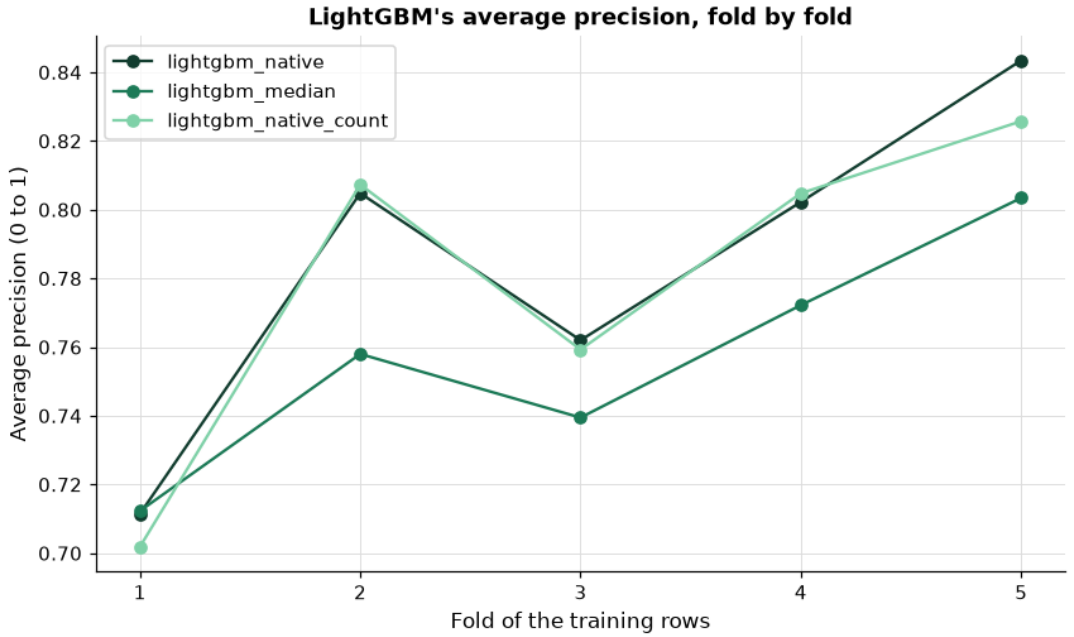
raw, plus the count

Average precision, mean of five folds; fold standard deviation near 0.0448. LightGBM learns which side of each split the gaps go to; an imputer makes a gap look typical.

A description of the training rows only.

The count uses only the row's own cells, so it is a fair feature. The table uses the outcome, so it is not.

LightGBM fold by fold

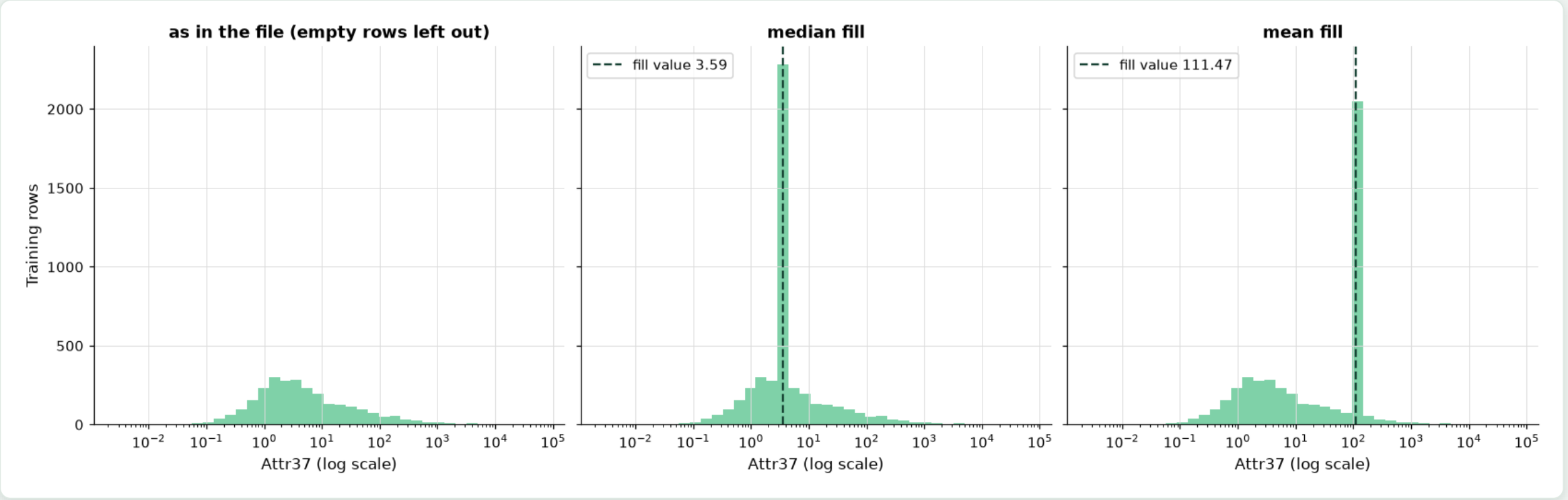


Model	AP	Fold sd	ROC AUC
lightgbm_native	0.7848	0.0448	0.9523
lightgbm_median	0.7571	0.0306	0.9502
lightgbm_native_count	0.7798	0.0447	0.9503

AP: average precision, five folds of the training rows. The lines move together from fold to fold far more than they move apart.

A difference smaller than the fold spread is not one these rows can show.

What a fill value does



2,000 of 4,680 rows (42.7%) are empty. Median fill 3.587, mean fill 111.470: the median looks typical, the mean looks like no real company.

How the transforms work

1. Standardize

mean 0 and spread 1, per column

2. Quantile

rank among training values, onto a normal curve

3. Impute and flag

median, plus a 0/1 column per gappy ratio

4. Learn inside

every number from the fitted rows

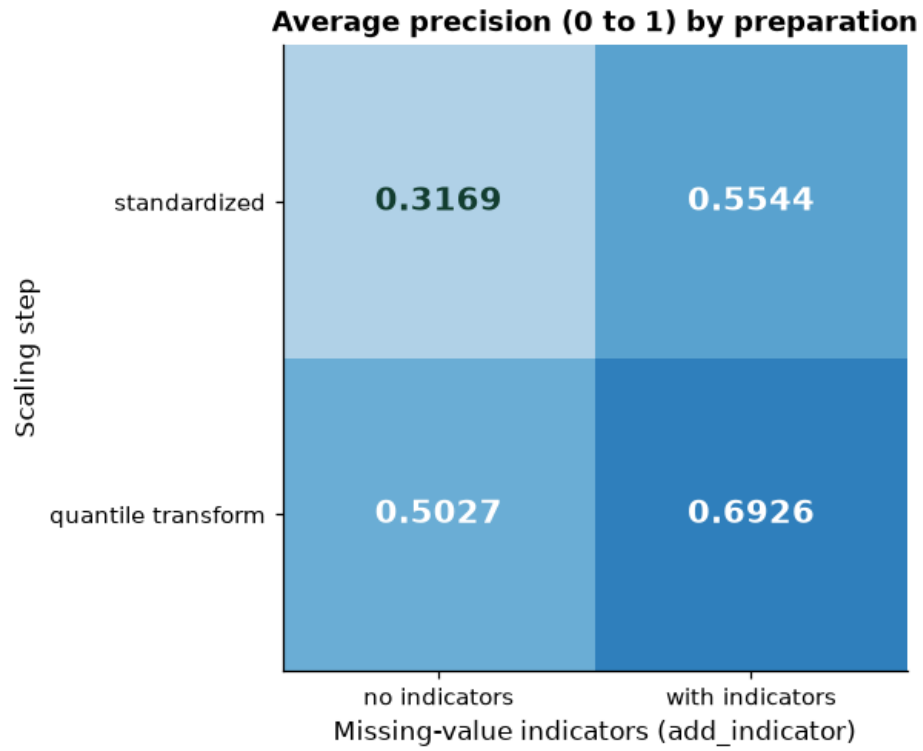
You control

The transform, the imputer's strategy, and the indicator.

What comes out

A prepared table, and the logistic regression's score.

Transforms for a linear model



Why it matters here

A straight-line model gives a far value a long lever, and cannot see a gap unless an indicator column says so. Each step lifts the score on its own; together they give the darkest cell.

Average precision, mean of five folds of the training rows: 0.3169 top left, 0.6926 bottom right.

Preparation more than doubles the score; LightGBM on raw columns is still at 0.7848.

How clipping works

1. Learn the bounds

1st and 99th percentile of the fitted rows

2. Clip

values beyond the bounds move to the bounds

3. Leave gaps empty

a missing value stays missing

4. Per fold

fit sees only the training folds

You control

The two percentiles.

What comes out

A table of the same shape with its tails pulled in; lower_ and upper_.

Clipping at the training rows' percentiles

X27 runs from -158,130.0 to 214,880.0. Its 1st and 99th percentiles, -77.6614 and 4,516.7060, move 44 values up and as many down; missing values stay missing.

X27 bounds	Lower	Upper
fold 1	-71.0499	5345.642
fold 2	-73.4695	4243.157
fold 3	-70.5788	5225.174
fold 4	-138.9016	3832.268
fold 5	-71.0727	4873.028

0.5544

standardized, indicators

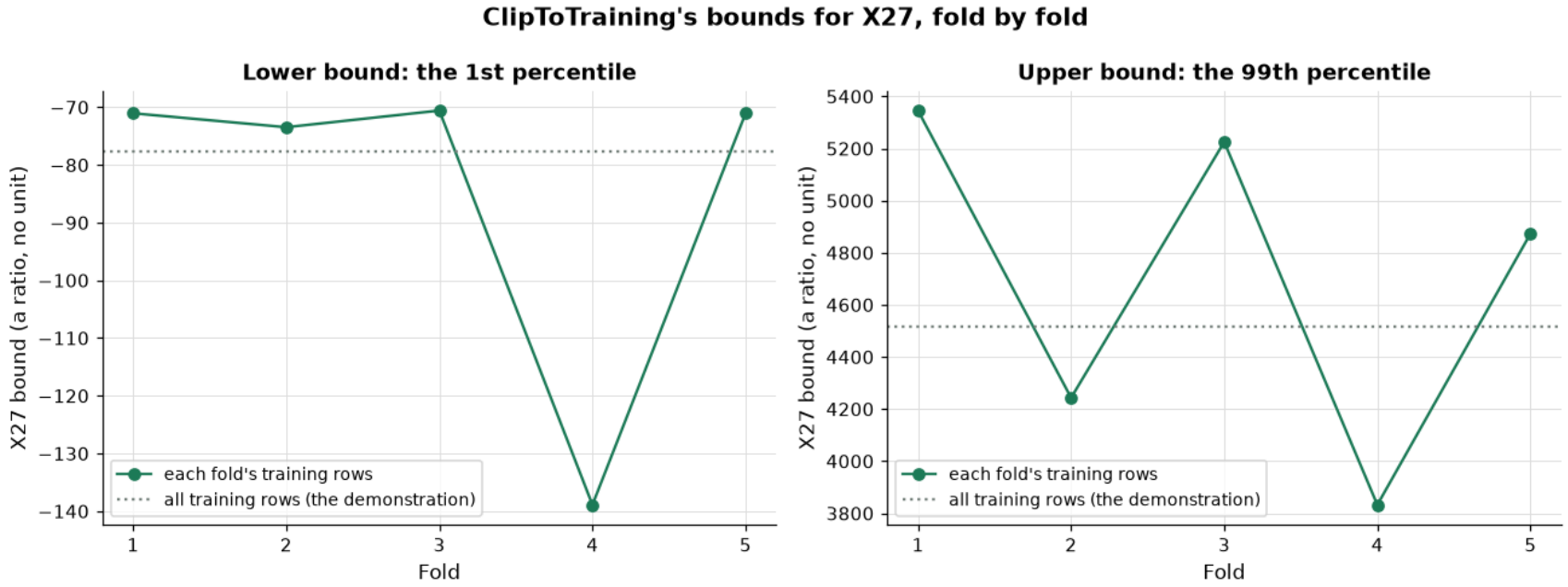
0.6530

clipped first

```
Pipeline([("clip", ensemblekit.ClipToTraining()), ...])
```

Five folds, five sets of bounds: each learned from its own training rows.

Each fold, its own bounds



A percentile out in a long tail rests on a few values, so it moves with the rows each fold holds: learn it inside the fold.

How a feature is tested

1. Same everything

nothing else changes

2. Add the feature

inside the pipeline, at the front

3. Score both

cross-validated, with and without

4. Read against sd

a gain counts when it exceeds the fold sd

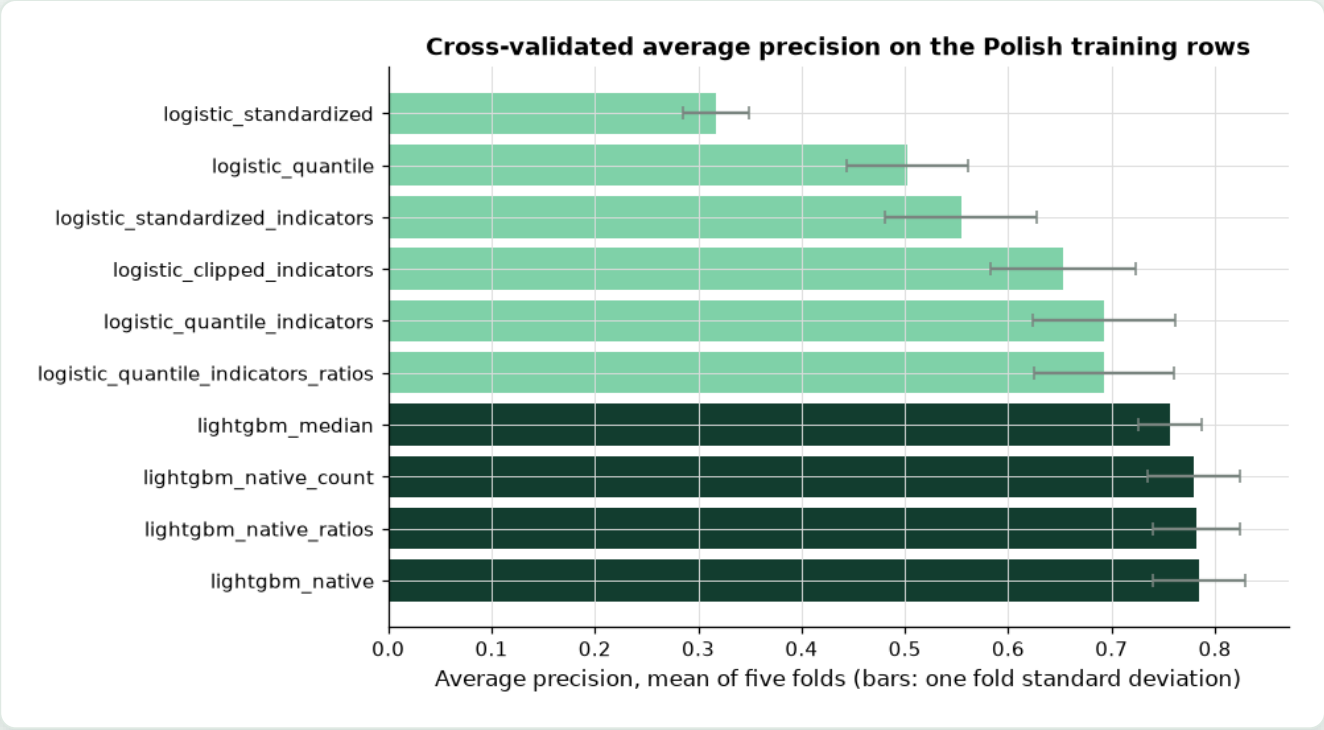
You control

The features added, and the model.

What comes out

A score with and without, for each model, and the fold sd.

What the new ratios add

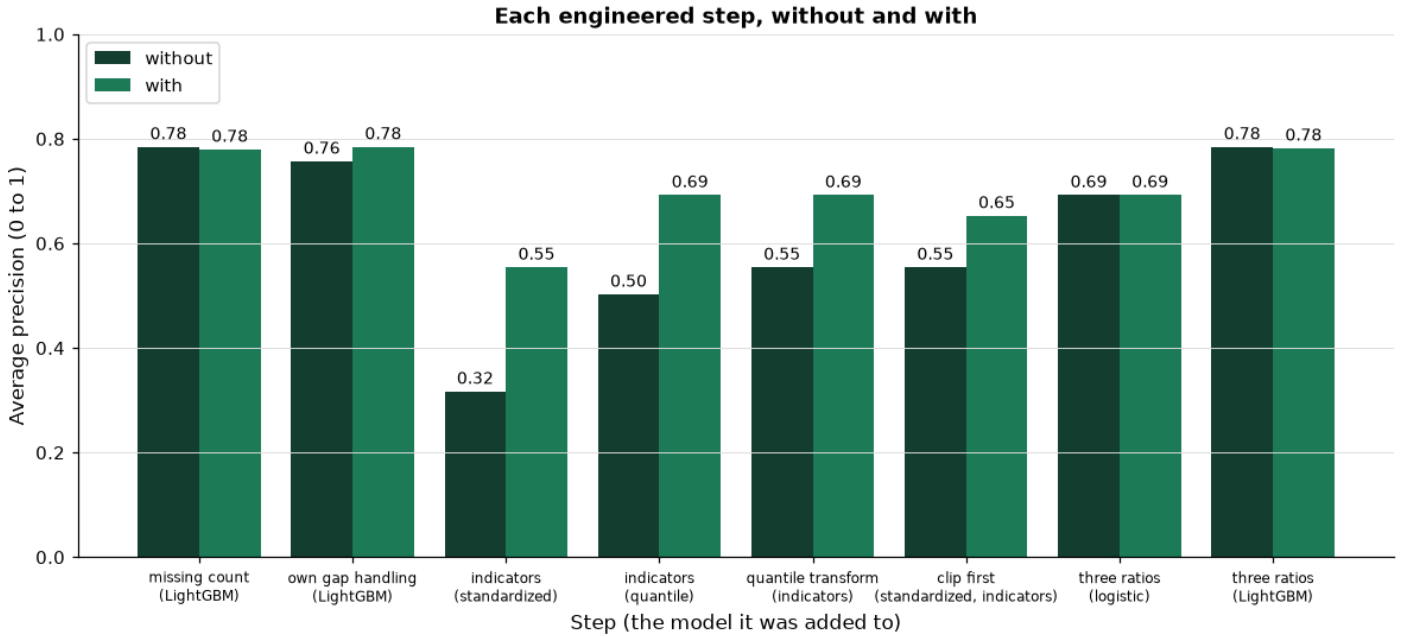


Model	Without	With ratios
Logistic	0.6926	0.6928
LightGBM	0.7848	0.7821

Both changes are a small fraction of a fold standard deviation: nothing measurable, on this file and these models.

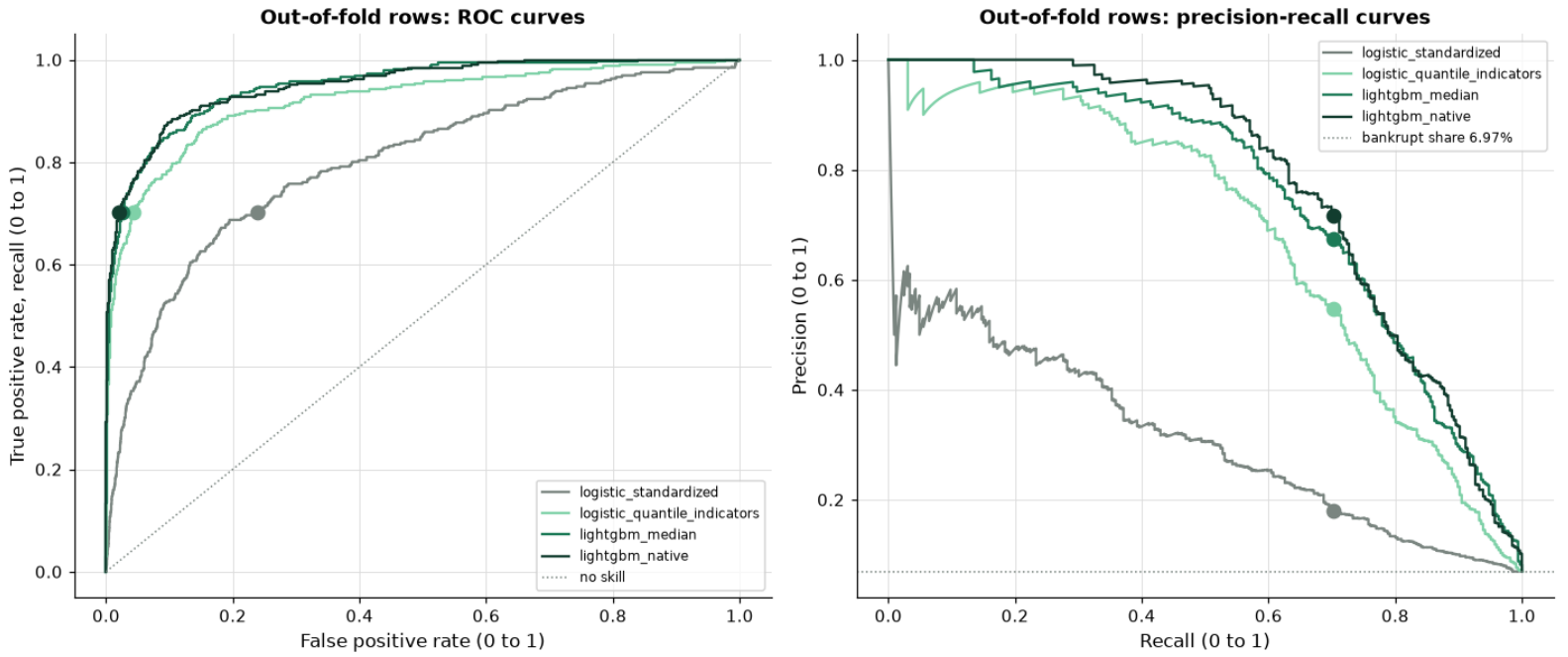
For the linear model the gain came from preparation, not from new columns.

Each step, without and with



Preparation steps add 0.10 to 0.24; the count and the ratios move their model by 0.0050 at most.

Four models, one set of axes



Every dot is at recall 0.7025; precision there: 0.1800 for the standardized logistic regression, 0.7156 for LightGBM.

Three kinds of feature, three rules

1. Row by row

The cap, the ratios, the count

Cannot leak. A FunctionTransformer keeps it with the model.

2. Learned from rows

Medians, indicators, quantiles, clip bounds

Inside the pipeline: each fold's training rows only.

3. From the outcome

A bankrupt share by group

The worst leak. Each fold's training rows only, if at all.

And no feature changes day 1's rule: Course 4's test rows are never scored.

Excel side by side

Excel side by side (tested in Excel)

`=MIN(A2,10)`	the cap, equal to numpy on 615 bonds
`=COUNTIF(A2:A616,">10")`	181 bonds changed
`=AVERAGE(ABS(EXP(C2:C616)-EXP(D2:D616)))`	40.9 bp, the capped model
`=IF(OR(B2=0,B2="",A2=""),NA(),A2/B2)`	#N/A on 14 rows, as safe_ratio
`=A2/B2`	#DIV/0! on 14; 0 for an empty numerator
`=PERCENTILE.INC(A2:A4681,0.01), 0.99`	-77.6614, 4516.7060
`=MIN(MAX(A2,$D$1),$D$2)`	an empty cell becomes the lower bound
`=COUNTBLANK(A2:BL2)`	the missing count, equal on 4,680 rows

Excel has no worksheet function for cross-validation, a pipeline, or boosting. Every feature built today is a formula it has.

Save the day: what the ratios added

1. Code

mlkit.py, ensemblekit.py, its tests

2. Log

Four models; features.txt so far

3. Ratios

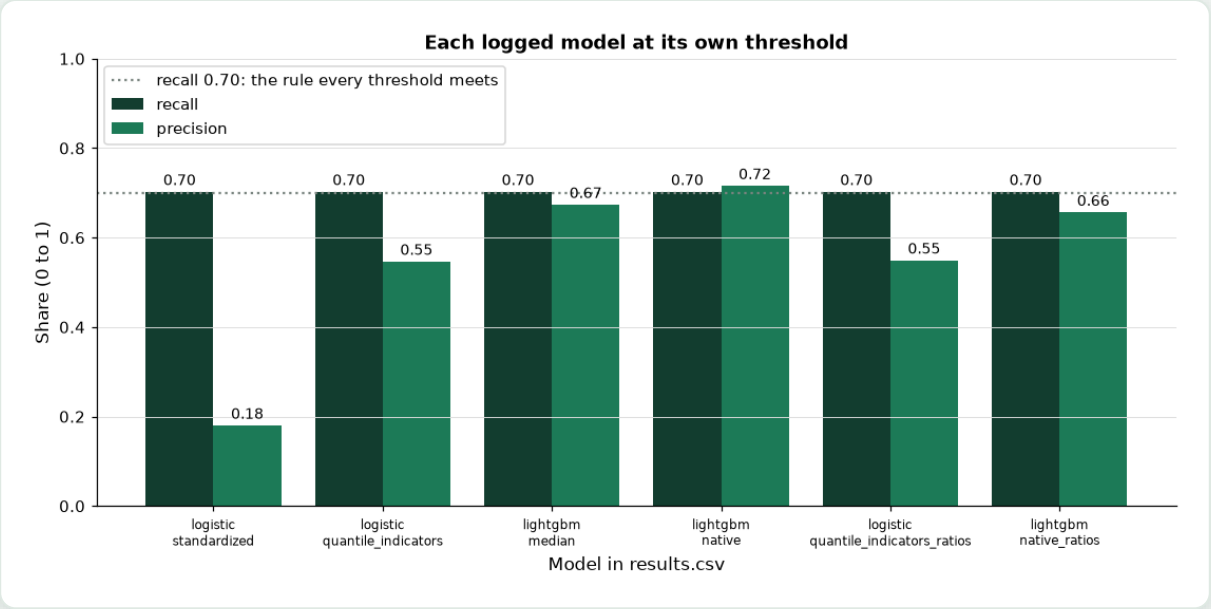
Two more models; three names added

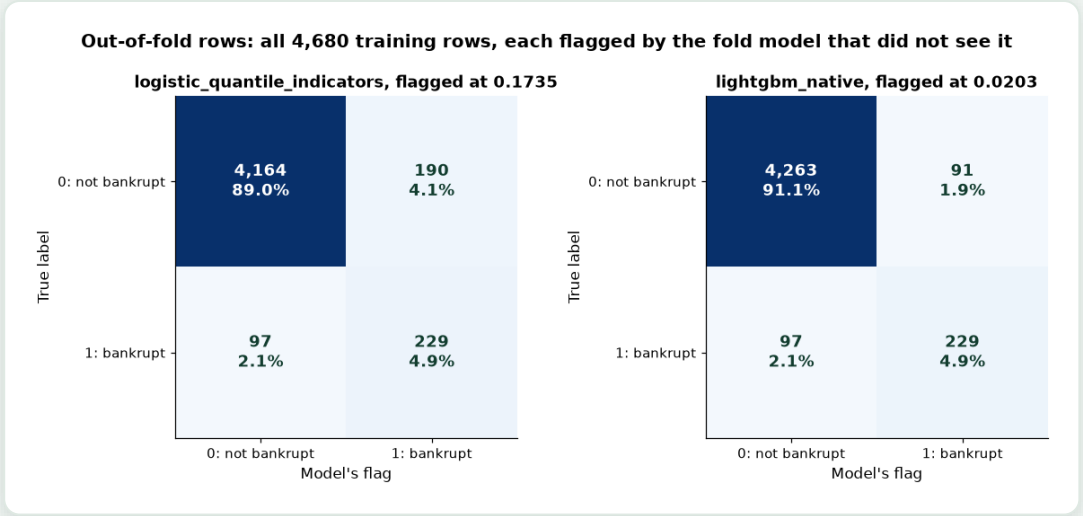
```
git diff HEAD~1 -- features.txt
+debt_to_equity
+ebit_to_liabilities
+short_term_share
```

```
model,rows,roc_auc,average_precision
logistic_quantile_indicators_ratios,cv,0.9208,0.6928
lightgbm_native_ratios,cv,0.9558,0.7821
```

Rows cv: fold means; threshold and flags from the pooled out-of-fold probabilities.

The log, drawn





Same recall, 0.7025, for every model; 229 of 326 bankrupt rows found. The top right cells are the companies flagged that did not go bankrupt.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: a pattern's bankrupt share learned from all rows before the folds: 0.7113, flattered.

Code that runs is not the same as code that is right.

What you can do now

- Write a known shape as a feature
- Build ratios that never return an infinity
- Use gaps: indicators for a linear model
- Transform or clip, learned inside the fold
- Sort every feature into one of three kinds

Week 3, Day 3 Resampling, SMOTE, and class weights